interrupt() method of Thread class :

It is a predefined non static method of Thread class. The main purpose of this method to disturb the execution of the Thread, if the thread is in waiting or sleeping state.

Whenever a thread is interrupted then it throws InterruptedException so the thread (if it is in sleeping or waiting mode) will get a chance to come out from a particular loop.

Points :-

If we call interrupt method and if the thread is not in sleeping or waiting state then it will behave normally.

If we call interrupt method and if the thread is in sleeping or waiting state then we can stop the thread gracefully.

*Overload interrupt method is mainly used to interrupt the thread safely so we can manage the resources easily.

Methods :-

1) public void interrupt() :- Used to interrupt the Thread but the thread must be in sleeping or waiting mode.

2) public boolean isInterrupted() :- Used to verify whether thread is interrupted or not.

//Programs :-

```
package com.ravi.interrupt;

class Test implements Runnable
{
@override
public void run()
{
    boolean interrupted = false;

    System.out.println("Is Thread Interrupted :"+interrupted);

    for(int i=1; i<=5; i++)
    {
        System.out.println("i is :"+i);

        try
        {
            Thread.sleep(1000);  //IllegalArgumentException
        }
        catch(InterruptedException e)
        {
            System.out.println("Child Thread Interrupted");
            Thread.currentThread().interrupt();  //isInterrupted = true;
        }
    }
}
}

public class InterruptDemo1
{
    public static void main(String[] args)
    {
        Test t = new Test();
        Thread th = new Thread(t);

        //main thread is interrupting the child thread
        th.interrupt();  //isInterrupted = true;
    }
}
```

Note :- In the above program main thread is interrupting child thread so interrupt flag value will become true, now since the child thread (th) is sleeping this program will get a message IllegalArgumentException and child thread will throw InterruptedException and interrupt flag value will become false

Note

```
package com.ravi.interrupt;

class Interrupt extends Thread
{
@override
public void run()
{
    Thread.currentThread().interrupt();  //self interruption

    for(int i=1; i<=5; i++)
    {
        System.out.println(i);

        try
        {
            Thread.sleep(1000);  //Java.lang.InterruptedException
                                 //interrupt flag value will become false
        }
        catch (InterruptedException e)
        {
            System.out.println("Thread has Interrupted");
            Thread.currentThread().interrupt();  //isInterrupt flag value will become true
        }
    }
}
}

public class InterruptDemo2
{
    public static void main(String[] args)
    {
        Interrupt i = new Interrupt();
        System.out.println("Thread State is :"+i.getState());  //NEW
        i.start();
    }
}
```

```
package com.ravi.interrupt;

public class InterruptDemo3
{
    public static void main(String[] args)
    {
        Thread thread = new ThreadDemo MyRunnable();
        thread.start();

        try
        {
            Thread.sleep(5000);  //Main thread is waiting for 5 sec
            Thread.interrupt();
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }
        thread.interrupt();
    }
}

class MyRunnable implements Runnable
{
@override
public void run()
{
    try
    {
        while (!Thread.currentThread().isInterrupted())
        {
            System.out.println("Thread is running by looking the resource");
            Thread.sleep(100);
        }
    }
    catch (InterruptedException e)
    {
        System.out.println("Thread interrupted gracefully");
    }
    finally
    {
        System.out.println("Thread resource can be release here");
    }
}
}
```

Deadlock :

```
==========

                T1                  T2
                 +                   +
                 |                   |
                R1                  R2
                 |                   |
                 +                   +
                R2                  R1
                 |                   |
                 +                   +
                END                 END
```

It is a situation where two or more more than two threads are in blocked state forever, here threads are waiting to acquire some resources without releasing any resources.

This situation happens when multiple threads demands same resource without releasing its own resources. Deadlock is a bug, it is very difficult situation and our evaluation of the program will go to infinite state as shown in the diagram

```
package com.ravi.interrupt;

class DeadLockDemo1e
{
public static void main(String[] args)
    {
        String resource1 = new String("Resource1");  //Obj1
        String resource2 = new String("I R Ragav");  //Obj2

    // I1 tries to lock resource1(obj1) then resource2(obj2)

        Thread t1 = new Thread();
        t1.start();
    @override
    public void run()
    {
        synchronized (resource1)
        {
            System.out.println("Thread 1: locked resource 1");
                try
                {
                    Thread.sleep(100);
                }
                catch (Exception e)
                {
                }
    //t1 thread is waiting here for lock2
            synchronized (resource2) //Wanted synchronized block
            {
                System.out.println("Thread 1: locked resource 2");
            }
        }
    }
    };

    // I2 tries to lock resource1 then resource2
    Thread t2 = new Thread();
    @override
    public void run()
    {
        synchronized (resource2)
        {
            System.out.println("Thread 2: locked resource 2");
                Thread.sleep(100);
                catch (Exception e)
                {
                }
    //t2 thread will wait for L1 (Resource1)
            synchronized (resource1) //Wanted synchronized block
            {
                System.out.println("Thread 1: locked resource 1");
            }
        }
    }
    };
    t2.start();
    t1.start();
    }
}
```

*** Issues extends Thread and implements Runnable which one is better and why ?

Ans :- Both are the ways in Java.lang package to create a Thread.
    Extends Thread is a class and implements Runnable is an interface.
    Runnable is more better approach due to the following reasons :

    a) In extends Thread we cannot extend any other class (MI) is not supported by using class but in implements Runnable we can extend any other class.

    b) extends Thread provides tight coupling approach so to create multiple threads we need multiple class objects but in implements Runnable provides loose coupling approach so for creating multiple threads we need only one class object.

    Example :
    Case 1 :

    class Test extends Thread
    {
    }

    Test t1 = new Test();
    t1.start();
    Test t2 = new Test();
    t2.start();

    Case 2 :

    class UserThread implements Runnable
    {
    }

    UserThread ut = new UserThread();

    Thread t1 = new Thread(ut);
    Thread t2 = new Thread(ut);

    2) By using extends Thread we can't use Lambda Expression but using implements we can use Lambda Expression.

    c) In class extends Thread then all the Thread class methods are available in the sub class but in implements Runnable then Thread class methods are not available.

---

volatile keyword in Java :- Thread may cache the variable from the main memory not from the cache memory

While working in a multithreaded environment multiple threads can perform read and write operation with common variable(data) chances of Data inconsistency so use synchronized OR volatile keyword concurrently.

In order to store the value temporarily, every thread is having local cache memory JM, whenever a thread wants to read a variable it will read from main memory but thread's value is not stored in thread's local cache; it is always read from the main memory.

So the conclusion is, a volatile variable value is always read from and written directly to main memory, which ensures that changes made by one thread are visible to all other threads immediately.

```
package com.ravi.interrupt;

class ShareData
{
    private volatile boolean flag = false;//FLAG VALUE WILL COME FROM MAIN MEMORY

    public void startThread()
    {
        Thread writer = new Thread();//
            @override
            public void run()
            {
                System.out.println("Writer thread started");
                Thread.sleep(5000);  //writer thread will go for 5 sec waiting state
                flag = true;
                System.out.println("Writer thread make the flag value as true");
            }
        catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        };

        Thread reader = new Thread();//

        while (!flag);//Cache memory still the value of flag is false
            {
            }
        System.out.println("Reader thread got the updated value");
        };

        writer.start();
        reader.start();
    }
}

public class VolatileExample
    {
        public static void main(String[] args)
            {
                new ShareData().startThread();
            }
    }
```

Remaining Methods Of Object class.

1) protected native Object clone() throws CloneNotSupportedException :

It is a predefined non static method of Object class.

It is mainly used to create a duplicate copy of original object.

We can perform Deep clone operation, if the object is Cloneable object that means that particular class must implements java.lang.Cloneable interface which is a marker interface.

It is a protected method so we need to override the method and called the Object class clone() method.

It throws a checked exception i.e. java.lang.CloneNotSupportedException
so handling is compulsory.

It follows deep copy concept so if we modify one object then another object remains unchanged.

clone() is method of Object class but not Cloneable interface, It is marker interface.

Steps we need to follow to perform clone operation :

1) The class must implements Cloneable interface
2) Override clone method from Object class (Because OM by catch)
3) In the Overridden method call Object class clone method [super.clone()]
4) Handle CloneNotSupportedException (Because OM throws exception)
5) Two different objects are created (deep copy).

//Program how to create Duplicate Object for Customer

```
package com.ravi.clone;

class Customer implements Cloneable
{
    private Integer id;
    private String name;

    public Customer(Integer id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    public void setId(Integer id)
    {
        this.id = id;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    @override
    public Object clone() throws CloneNotSupportedException
    {
        return super.clone();
    }

    @override
    public String toString()
    {
        return "Customer [id= " + id + ", name= " + name + "]";
    }
}

public class CustomerCloneDemo
{
    public static void main(String[] args)
    {
        Customer c = new Customer(101, "Sohit");
        System.out.println(c);

        try
        {
            Customer c1 = (Customer) c.clone();
            System.out.println(c1);
            System.out.println("After Modification Object c1 data");
            c1.setId(200);
            c1.setName("Raj");

            System.out.println(c);
            System.out.println(c1);
        }

        catch(CloneNotSupportedException e)
            {
                System.out.println("Object cloning is not possible :"+e);
            }
    }
}
```

Note : The main purpose of Marker interface is to provide additional information regarding Object like Object is Cloneable OR Serializable OR Randomly Accessible. Without these information, JVM will not allow to perform operation on these objects.

---

protected void finalize() throws Throwable

```
Product p1 = new Product(111,"LaptopC",123000);
    p1.removeProductDataFromDB();
    p1 = null;

p1 ---------->  +-----------+
                |  Product  |
                |  Object   |
                +-----------+                    Database
```